

INSIGHTFLOW AI

Schema-Aware Business Intelligence & Predictive Analytics Platform

Document: Software Requirements Specification (SRS)

Version: 1.0.0

Project Scope: Academic Minor Project (MCA Semester 3)

Author: Saurabh Sahu (MCA III Sem)

Institution: Department of Computer Applications

Date: July 9, 2026

1. Introduction

1.1 Purpose of the Document

This Software Requirements Specification (SRS) document details the complete functional requirements, non-functional constraints, data structure dictionaries, and architecture principles for the **InsightFlow AI** analytics suite. It serves as a single source of truth for design, implementation, and academic evaluation of this Semester 3 Minor Project.

1.2 Product Scope

InsightFlow AI is a high-performance business intelligence (BI) system. Unlike standard BI systems, it features dynamic schema-agnostic database ingestion, vectorized online analytical processing (OLAP) querying, automatic heuristic-based data cleaning, and statistical machine learning (time-series forecasting and customer segmentation). The system consists of a FastAPI service backed by SQLite/PostgreSQL (for metadata ledger) and DuckDB (for analytical processing), alongside a React+Vite web frontend presenting custom dark-themed dashboards.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
OLTP	Online Transaction Processing — database system optimized for ledger entries (PostgreSQL/SQLite).
OLAP	Online Analytical Processing — optimized for fast aggregates and vectorized multi-dimensional queries (DuckDB).
RBAC	Role-Based Access Control — user permissions segmented by organizational roles.
ETL	Extract, Transform, and Load — pipeline to clean, map, and import flat files.
Holt-Winters	A triple-exponential smoothing statistical algorithm used for additive/multiplicative forecasting.

2. Overall Description

2.1 Product Perspective

InsightFlow AI operates as a split-architecture fullstack system. The React frontend consumes RESTful JSON endpoints exposed by the FastAPI server. Data is stored across two separated database engines: a transaction registry (PostgreSQL/SQLite) and a dedicated query engine (DuckDB).

2.2 Product Functions

The platform provides five primary product pillars:

- **Ingestion Engine:** Handles CSV or Excel uploads of varying structures, parses headers, and performs semantic typing.
- **ETL Clean Pipeline:** Detects and fixes missing values (imputation via median/mode), standardizes currencies, removes duplicates, and marks statistical outliers.
- **OLAP Analytics:** Generates rapid multi-dimensional charts (revenue timelines, regional maps, payment methods, category breakdowns) using DuckDB.
- **Forecasting Engine:** Calculates historical growth rates and projects future revenues using triple exponential smoothing.
- **Reporting Engine:** Generates downloadable PDF briefs and multi-tab Excel workbooks containing detailed operational audit summaries.

2.3 User Classes & Roles

The system features a strict hierarchical RBAC structure mapping to system privileges:

Role	Privilege Scope
CEO	Full system access. Can view global charts, ingest new datasets, export reports, and view audit trails.
Store_Manager	Can view regional and product charts. Authorized to download report briefs.
Sales_Manager	Authorized to look up products and customer segment analytics.
Data_Analyst	Ingests raw data files, executes the ETL cleaning runner, and checks logs.

3. Functional Requirements

3.1 Ingestion and Domain Classification

Upon file upload, the system must parse columns and match keywords against predefined dictionary groups to compute matching scores for the following business domains: **InsightFlow (Retail/E-Commerce)**, **Finance**, **HR**, **Healthcare**, and **Marketing**. The domain with the highest scoring matches is dynamically selected, and the confidence metric is recorded in the metadata ledger.

3.2 Dynamic ETL Data Cleaning Heuristics

The system must execute these cleaning steps when triggered:

1. **Duplicate Removal:** Deduplicate rows based on identical column values.
2. **Mode Imputation:** Impute missing string values using the most frequent value in the column.
3. **Median Imputation:** Impute missing numerical cells using the median value.
4. **Numeric Extraction:** Strip currency symbols (\$, €, etc.) or formatting characters, converting values to floats.
5. **Outlier Flagging:** Identify values outside $1.5 * IQR$ (Interquartile Range) boundary limits.

3.3 Vectorized Query Engine (OLAP)

The system must initialize an in-memory DuckDB query session directly over the cleaned pandas dataframes. It must execute multi-dimensional aggregate queries (grouping by date, product category, store branches, and payment methods) and return responses in less than 200 milliseconds to populate frontend analytics dashboards.

3.4 Machine Learning & Forecasting

The platform must construct dynamic time-series datasets of transaction aggregates. It must train a triple-exponential smoothing model (Holt-Winters method) on historical data, computing level, trend, and seasonal components, and output a 30-day forecast bound with confidence intervals.

3.5 Exporter Service

Users must be able to export audit briefs as a formatted PDF (using ReportLab) or a multi-sheet Excel file (using OpenPyXL) displaying details of ingestion logs, cleaning results, and transactional aggregates.

4. Database Schema (3NF)

The relational database schema is normalized to Third Normal Form (3NF) to guarantee transactional integrity. It contains 15 core tables:

Table Name	Key Attribute Columns
------------	-----------------------

roles	role_id (PK), role_name, description
users	user_id (PK), email, password_hash, first_name, last_name, role_id (FK), is_active
stores	store_id (PK), store_code, store_name, region, city, state
categories	category_id (PK), category_name, parent_category_id (FK)
products	product_id (PK), sku, product_name, category_id (FK), cost_price, insightflow_price
customers	customer_id (PK), customer_code, first_name, last_name, email, segment, registration_date
orders	order_id (PK), order_number, customer_id (FK), store_id (FK), order_date, total_amount
order_items	order_item_id (PK), order_id (FK), product_id (FK), quantity, unit_price, net_amount
inventory	inventory_id (PK), product_id (FK), store_id (FK), quantity_on_hand, safety_stock, reorder_point, supplier_id (FK)
suppliers	supplier_id (PK), supplier_name, contact_name, email, phone
employees	employee_id (PK), first_name, last_name, store_id (FK), email, role, salary
payments	payment_id (PK), order_id (FK), payment_date, payment_method, amount, status
returns	return_id (PK), order_item_id (FK), return_date, quantity, refund_amount, return_reason
upload_history	upload_id (PK), filename, file_size, rows_count, columns_meta, domain, clean_status, created_at
audit_logs	log_id (PK), user_id (FK), action_type, description, ip_address, created_at

5. Non-Functional Requirements

5.1 Security Requirements

Passwords must be encrypted using **bcrypt** hashing. User sessions must be validated using cryptographically signed ****JSON Web Tokens (JWT)**** with HS256 encryption. Database connections must utilize parameterized queries to prevent SQL injections.

5.2 Performance & Scalability

OLAP query response times must remain sub-second (under 200ms) for aggregated reports of up to 100,000 transaction records, supported by DuckDB's vectorized columnar engine. The backend API must support asynchronous processing for large dataset uploads, freeing HTTP threads.

5.3 Portability & Robustness

If a centralized PostgreSQL database is unreachable, the system must automatically fallback to a localized SQLite file database (`insightflow_ai.db``) to guarantee offline capabilities. File cleaning routines must handle corrupt inputs gracefully, logging anomalies without crash failures.

5.4 Usability & Aesthetics

The UI must employ modern design aesthetics, prioritizing a premium dark theme, cohesive glassmorphic layouts, and clear typographic contrast (using custom font scaling via Tailwind CSS). Dashboards must adjust to mobile and desktop screens dynamically.